Day 7 — Bootloaders, OTA & System Design

Bootloader Architecture, OTA Updates & Capstone Project

1 | What Is a Bootloader?

A bootloader is the first code that runs after reset. It validates, selects, and launches the main application. On embedded MCUs it lives at the lowest flash address (e.g. 0x08000000 on STM32) and is typically the smallest, most trusted piece of firmware on the device.

Primary bootloader	Factory-burned ROM loader (ST's built-in UART/USB DFU). Cannot be erased.
Secondary bootloader	Your custom code in flash — handles OTA, signature checks, slot switching.
Application	Main firmware loaded and jumped to by the bootloader.
Reset vector	Stack pointer + reset handler address at base of each image (offset 0 & 4).
Vector table offset	SCB->VTOR must be set to the application's flash base before jumping.
Watchdog in BL	Always kick or disable WDT before long erase/write operations.

2 | Flash Memory Layout

Plan your flash map before writing a single line of bootloader code. A typical dual-bank OTA layout for STM32L4 (512 KB flash):

Region	Address	Size	Contents
Bootloader	0x08000000	32 KB	BL code + vector table. Write-protected in production.
Boot config	0x08008000	4 KB	Flags: active slot, update pending, boot count.
Slot A (active)	0x08009000	236 KB	Current running application image.
Slot B (update)	0x08044000	236 KB	Newly downloaded OTA image awaiting validation.
NVM / EEPROM	0x0807C000	4 KB	Device settings, calibration, runtime state.

Tip: store the boot config in a dedicated page so you can erase/rewrite flags without touching application code.

3 | Bootloader Startup Sequence

Step	Action	Notes
1	Hardware init	Clocks, UART, WDT, GPIOs — absolute minimum only.
2	Read boot config flags	Check: update_pending, slot_valid[A/B], boot_count.
3	Update pending?	Copy Slot B to Slot A (or swap pointers in dual-bank HW).
4	Validate active image	CRC32 or SHA-256 over image; check magic number + length.
5	Signature verify (secure BL)	Verify ECDSA-P256 signature against public key in BL.

6	Boot count guard	If boot_count > 3 and app keeps crashing, revert to Slot A backup.
7	Set VTOR	SCB->VTOR = APP_FLASH_BASE; — redirect interrupt vectors.
8	Jump to application	Load SP from word 0, jump to reset handler at word 1.

// Minimal jump-to-application in C

```
typedef void (*pFunc)(void);
uint32_t *appVectors = (uint32_t*)APP_FLASH_BASE;
__set_MSP(appVectors[0]); // set stack pointer
SCB->VTOR = APP_FLASH_BASE; // redirect vectors
pFunc jumpToApp = (pFunc)appVectors[1];
jumpToApp(); // never returns
```

WARNING: Disable all interrupts and peripherals before jumping. An active DMA or timer ISR firing during the jump causes a hard fault.

4 | Image Validation — CRC & Signatures

Magic number	4-byte value at fixed header offset — quick sanity check (e.g. 0xDEADC0DE).
Image length	Stored in header; bounds-check before CRC to avoid reading past flash end.
CRC-32	Fast hardware CRC on STM32 (CRC unit). Detects corruption, not tampering.
SHA-256 hash	Cryptographic integrity; use mbedTLS or hardware HASH peripheral.
ECDSA-P256 sig	Asymmetric signature; private key signs on build server, BL verifies with public key.
Version field	Reject downgrades: refuse any image with version less than current.
Rollback counter	Monotonic counter in OTP/eFuse prevents flashing old signed images.

// STM32 hardware CRC-32 over image (HAL)

```
MX_CRC_Init();
uint32_t crc = HAL_CRC_Calculate(&hcrc;, (uint32_t*)APP_BASE, img_words);
if (crc != img_header.crc32) { /* mark invalid, revert */ }
```

5 | OTA Update Architectures

Strategy	Flash Required	Pros	Cons
Dual-bank swap	2x app size	Atomic swap; instant rollback.	Needs 2x flash space.
Download-and-overwrite	1x app + scratch	Less flash needed.	Power-loss during write = brick risk.
Delta / diff OTA	Varies	Tiny update packets over slow links.	Complex patching; more BL code.
External flash staging	Ext. SPI flash	App flash untouched during download.	Requires SPI flash chip.
Encrypted OTA	Any of above	Image confidential in transit and at rest.	Need key management strategy.

Transport options	UART, USB DFU, BLE, Wi-Fi MQTT/HTTPS, LoRaWAN fragmented block transfer.

Update server	AWS IoT Jobs, Azure Device Update, custom HTTPS endpoint, or local TFTP.
Chunk size	Match to flash page size (e.g. 2 KB) to avoid partial-page writes.
Progress tracking	Store last written offset in boot config; resume after power loss.

6 | Secure Boot & Root of Trust

Secure boot ensures only authenticated firmware runs on the device, protecting against malicious firmware injection and supply-chain attacks.

RDP (Read Protection)	STM32 RDP level 1/2 prevents flash readout via debug interface.
Option bytes	OB_WRP write-protects bootloader pages from accidental erasure.
TrustZone (ARMv8-M)	Cortex-M33: BL in Secure world; app in Non-Secure world; hardware enforced.
Unique device ID	96-bit UID at 0x1FFF7590 — use to bind firmware licenses or keys.
mbedTLS	Lightweight TLS 1.3 + ECDSA/AES for BLE/Wi-Fi OTA channels.
MCUboot	Open-source secure bootloader; widely used with Zephyr & TF-M.

7 | System Design — Embedded Architecture Patterns

Pattern	Description	Best for
Super-loop	Single while(1) polling loop; no OS.	Ultra-simple, single-task devices.
Interrupt-driven	Main loop sleeps; ISRs handle all events.	Low-power, event-sparse systems.
Layered (HAL/BSP)	Hardware abstraction layer separates drivers from logic.	Portable, testable firmware.
State machine	Explicit states + transitions; handles complex event sequences.	Protocols, UI flows, motor control.
Active objects	Each object has private queue + task; no shared state.	Complex concurrent systems.
Event-driven + RTOS	Tasks communicate via queues/semaphores; no polling.	Most production IoT firmware.

8 | Capstone Project — IoT Sensor Node

Design goal: a battery-powered BLE sensor node that reads temperature/humidity every 60 s, advertises data over BLE, supports OTA firmware updates, and runs for 1+ year on a CR2032.

Hardware block diagram:

Block	Component	Interface	Notes
MCU + Radio	nRF52840 or STM32WB55	—	Cortex-M4 + BLE 5.2 SoC.
Sensor	SHT40 (temp/humidity)	I2C	3.3 V, 200 µA active, 0.1 µA sleep.
Storage	4 MB SPI NOR flash	SPI	OTA image staging buffer.
Power	CR2032 + 3.3 V LDO	—	220 mAh; target avg current 2 µA.

Debug	SWD + UART log	SWD/UART	Disable in production build.

Firmware task breakdown (FreeRTOS):

Task	Priority	Period	Responsibility
SensorTask	2	60 s	Wake SHT40, read I2C, push data to queue, sleep sensor.
BLEAdvTask	3	On data	Update BLE advertisement payload with latest readings.
OTATask	1	On cmd	Receive BLE OTA chunks, write to SPI flash, signal BL.
WatchdogTask	4	500 ms	Kick IWDG; escalate if any task misses its deadline.
Idle / tickless	0	—	FreeRTOS tickless idle; MCU in Stop 2 between events.

Power budget calculation:

Phase	Current	Duration / 60 s cycle	Charge (µAh/cycle)
BLE advertise	5 mA	10 ms	0.014
Sensor read	300 µA	50 ms	0.004
OTA (rare)	8 mA	avg 0 ms	~0
Stop 2 sleep	3 µA	~59.94 s	0.050
Total average	**~4.6 µA**	—	**~0.068**

Estimated runtime: 220 mAh / 0.0046 mA = ~47,800 hours = ~5.5 years on a CR2032.

9 | Production Readiness Checklist

Flash protection	Enable RDP Level 1; write-protect bootloader pages.
Debug disabled	Remove UART logs; disable SWD in production build or via RDP.
Watchdog enabled	IWDG running; WatchdogTask kicks it; timeout triggers reset.
Secure OTA	ECDSA signature verification before any image is accepted.
Anti-rollback	Monotonic version counter; reject older signed images.
Fault handlers	HardFault/BusFault handlers log PC+LR to NVM before reset.
Assert strategy	configASSERT logs file+line then triggers WDT reset in prod.
Unique serial number	Burn UID or custom serial to NVM at factory; include in BLE adverts.
Regulatory (FCC/CE)	BLE module must have certification; check antenna placement rules.
Field diagnostics	Expose firmware version, uptime, error log over BLE or UART service.

10 | Tools, Frameworks & Ecosystem

Category	Tool / Framework	Use

IDE	VS Code + CMake / STM32CubeIDE / IAR	Build, flash, debug.
Debugger	OpenOCD + GDB / J-Link / STLink	SWD/JTAG debugging.
RTOS	FreeRTOS / Zephyr / ThreadX	Task scheduling.
Secure BL	MCUboot / TF-M	Signed image boot + OTA.
TLS/Crypto	mbedTLS / WolfSSL	OTA transport security.
Static analysis	Cppcheck / PC-lint / Polyspace	MISRA-C compliance.
Unit testing	Unity + CMock / Google Test	Off-target firmware tests.
CI/CD	GitHub Actions + west / west flash	Automated build + flash.
Power profiling	Nordic PPK2 / Otii Arc	Current measurement.
Protocol testing	Wireshark / nRF Sniffer / Logic 2	BLE/UART/SPI analysis.

COURSE COMPLETE — All 7 Days Done!

7-Day Course Roadmap:

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 ✓	Interrupts, Timers & PWM
Day 3 ✓	UART, SPI, I2C — Deep Dive
Day 4 ✓	ADC/DAC & Sensor Interfacing
Day 5 ✓	RTOS Fundamentals (FreeRTOS)
Day 6 ✓	Power Management & Low-Power Design
Day 7 ✓	**Bootloaders, OTA & System Design Project**